REGULATORY CHANGE IMPACT ENGINE

A practical novelty blueprint for SIH Problem Statement 26262
Integrated AICTE Approval Workflow Automation, Compliance Monitoring and Alert Management System

Core idea

Turn an official regulatory update into a traceable, reviewable impact assessment: identify which requirements, workflows, evidence items, deadlines and active applications may be affected—then route proposed changes for authorised approval.

CONCEPT NOTE • MVP DESIGN • TECHNICAL PLAN • DEMO STORY • RISKS & SAFEGUARDS

Prepared as a hackathon implementation guide. Regulatory examples in this document are illustrative; the live system must use verified official notices and authorised interpretations.

1. Executive summary

The Regulatory Change Impact Engine (RCIE) is a change-management layer for a regulated approval workflow. When a new AICTE Approval Process Handbook (APH), circular, amendment, clarification or deadline notification is published, RCIE helps an authorised team answer five operational questions: What changed? Which configured rules might be affected? Which workflows and evidence requirements depend on those rules? Which active or upcoming applications may be impacted? What actions should be reviewed, communicated and tracked?

RCIE is not simply a news feed or notification system. Its value is the traceable link from a source document to a versioned rule, from that rule to the affected process elements, and from those dependencies to a human-reviewed impact list and action plan.

At a glance	Definition
Problem	Regulatory updates can be difficult to translate consistently into operational changes across forms, checklists, evidence requirements, deadlines, and in-progress cases.
Users	Authorised regulatory/policy administrators, approval operations teams, application reviewers, institutional compliance users, and auditors.
Primary output	A versioned change record, proposed rule differences, impact graph, affected-case shortlist, review tasks, notifications, and audit trail.
Novelty	Dependency-aware impact analysis with explainable links back to source clauses and controlled activation—not just storing circulars or sending reminders.
MVP principle	Start with structured, human-entered rule changes and deterministic dependency matching. Add AI-assisted extraction only as a draft-making aid.

One-line pitch

“RCIE turns regulatory updates into explainable, version-controlled change plans—showing what may be affected, why it is affected, and who must review the change before it becomes active.”

2. The problem and why it matters

A regulatory update is often published as a document, but implementing it can require coordinated changes across many system components. A revised eligibility criterion might affect an application checklist, a required evidence item, a validation rule, a reviewer task, a deadline, a notification template, and existing applications. Updating only one of these can create inconsistent instructions or missed obligations.

- **Fragmented interpretation:** teams may manually compare new and old documents and create separate action lists.
- **Hidden dependencies:** a change to one clause may affect multiple forms, evidence requirements, process stages, user roles, or deadlines.
- **Unclear applicability:** not every rule change applies to every institution, course, application type, or application date.
- **Weak traceability:** it may be difficult to show which official source justified a rule configuration or why a case was marked as affected.
- **Risk of premature rollout:** unreviewed interpretations can incorrectly change a statutory workflow or send misleading alerts.

RCIE addresses the operational translation and traceability problem. It does not decide what a regulation legally means; that interpretation remains with authorised people.

3. Proposed system and main components

Component	Purpose	MVP behaviour
1. Source & notice register	Record official regulatory documents and their provenance.	Capture title, issuing authority, publication/effective dates, official URL or file, document checksum, and status.
2. Change intake & comparison	Represent what is new, modified, removed, or clarified.	Author enters a proposed change; system compares structured old/new values and stores a draft.
3. Versioned rule registry	Keep current and historical configurations.	Each rule has an ID, source clause, version, validity window, scope, and approval status.
4. Dependency graph	Connect rules to system elements.	Map rules to forms, evidence types, validations, workflows, deadlines, templates, and application categories.
5. Impact analyser	Find potentially affected items and cases.	Traverse dependencies and filter applications by applicability and relevant dates; provide reasons for each match.
6. Review & approval workflow	Prevent unauthorised activation.	Route proposed changes to designated reviewers; support approve, reject, request clarification, or defer.
7. Action & alert manager	Turn impact results into tracked work.	Create tasks, due dates, targeted notices, acknowledgements, and escalation records.
8. Audit & rollback	Make the change history reconstructable.	Record who changed what, when, why, based on which source, approvals, and activation/rollback events.

4. End-to-end workflow

1. **Capture the source.** An authorised administrator records an official circular/handbook amendment, source URL or file, issue date, effective date if stated, and issuing authority.
2. **Create a draft change.** A human enters the affected rule and proposed old/new values. Optional AI/OCR can suggest candidate clauses or differences, but the suggestion remains unapproved.
3. **Map dependencies.** The rule registry links the affected rule to evidence requirements, application types, form fields, validations, workflow stages, deadlines, and notification templates.
4. **Run impact analysis.** RCIE finds dependent system elements and filters cases using explicit applicability conditions such as application type, institution category, course, state, submission date, or effective date—only where those attributes are known and configured.
5. **Explain each result.** Every flagged item shows the source rule, dependency path, matching condition, and reason it was included. Uncertain matches are marked for review rather than treated as facts.
6. **Authorised review.** Designated policy/operations personnel inspect the proposed interpretation and impact list, resolve ambiguities, and approve or reject the change.
7. **Controlled activation.** After approval, the new rule version is activated for the correct effective window. Historical versions remain available.
8. **Communicate and track.** The system creates targeted tasks and notifications, captures acknowledgements where useful, and tracks completion.
9. **Audit or reverse.** The audit trail records the lifecycle. If a configuration error is found, an authorised administrator can supersede or roll back the activation with a documented reason.

Important: “affected” means “potentially affected according to the configured rule and case data,” not “legally non-compliant.” The user interface must preserve this distinction.

5. Example scenario (illustrative only)

Suppose an official update changes a document requirement for a particular category of application. The numbers and requirement below are fictional and are used only to explain the workflow.

Step	Example engine output
Source	Circular C-2026-XX; source file and publication date stored; effective date awaits authorised confirmation.
Draft change	Evidence requirement E-14 changes from “document A” to “document A plus declaration B” for the specified application category.
Dependency path	Rule R-14 → evidence requirement E-14 → application checklist and upload page → pre-submission validation → reviewer checklist.
Potentially affected cases	18 draft/submitted cases match the configured application category and relevant date window. Each match has a reason and case-data snapshot.
Actions	Policy reviewer validates interpretation; operations owner reviews active cases; institutions receive a targeted request only after approved messaging.
Activation	New rule version becomes active on the approved effective date. Old cases follow the explicitly approved transition policy.
Audit	Source, draft values, dependency graph version, affected-case result, approvers, activation timestamp, and notifications are retained.

The demo should visibly show the chain: source document → proposed rule difference → dependency map → impacted applications → approval → targeted action.

6. Rule and dependency data model

Keep the data model explicit. The system should not infer regulatory applicability from a document alone or silently overwrite live rules.

Entity/table	Useful fields	Why it exists
regulatory_sources	id, title, issuing_authority, source_url, file_hash, published_at, effective_at, captured_at, status	Provenance and integrity of source documents.
regulatory_clauses	id, source_id, clause_ref, page_ref, extracted_text, verification_status	Traceable clause-level references; extraction is not itself an approved interpretation.
rules	id, rule_key, name, description, owner, domain, active_version_id	Stable identity for a regulatory requirement.
rule_versions	id, rule_id, version_no, structured_value, scope_json, valid_from, valid_to, source_clause_id, approval_status	Immutable history and effective-dated configuration.
change_proposals	id, source_id, summary, old_version_id, proposed_payload, rationale, status, created_by	Draft changes awaiting review.
dependencies	id, rule_version_id, target_type, target_id, relationship, applicability_expression	Links a rule to forms, evidence, workflows, deadlines, templates, or cases.
impact_runs	id, proposal_id, graph_version, started_at, completed_at, status, summary_json	Reproducible impact analysis runs.
impact_results	id, impact_run_id, target_type, target_id, impact_level, reason, dependency_path, confidence_or_review_flag	Stores each potentially affected target and explanation.
approvals	id, proposal_id, reviewer_id, decision, comments, decided_at	Formal review and approval history.
action_tasks	id, impact_result_id, owner_id, due_at, status, resolution_note	Operational follow-up.
notifications	id, task_id, recipient_id, channel, sent_at, delivery_status, acknowledgement_at	Targeted communication and delivery tracking.
audit_events	id, actor_id, event_type, entity_type, entity_id, before_hash, after_hash, timestamp, reason	Tamper-evident or append-only audit history.

Design notes

- Keep rule versions immutable; create a new version instead of editing a previously approved version in place.
- Represent effective dates and applicability scopes explicitly. A publication date and an effective date are not necessarily the same.
- Store the exact dependency path and rule/graph version used by every impact run so results can be reproduced.
- Keep source text, AI/OCR suggestions, human interpretation, and approved structured rule values as distinct records.
- Avoid a general-purpose expression evaluator that executes arbitrary code. Use a constrained, validated applicability format.

7. Impact analysis logic

For a hackathon MVP, a deterministic dependency traversal plus structured filters is more dependable and explainable than asking a language model to guess every affected application.

Simplified logic

```
1. Load proposal and current approved rule version.
2. Validate required fields and source references.
3. Find dependencies attached to the rule and its proposed version.
4. Traverse affected forms, evidence types, validations, workflows, deadlines and templates.
5. Select candidate cases by configured applicability and effective-date policy.
6. Record match reason, dependency path, data timestamp and unresolved assumptions.
7. Flag ambiguous or incomplete matches for human review.
8. Produce a draft impact report; do not activate rules or send external notices.
```

Impact categories

Category	Meaning	Suggested handling
Direct	The changed rule directly configures this field, evidence item, validation, or deadline.	Show exact dependency and proposed adjustment.
Downstream	The item depends on a component affected by the rule.	Show full dependency path and let reviewer confirm relevance.
Case-level candidate	A case matches configured scope and time rules.	Explain matching fields and applicability policy; do not label non-compliant automatically.
Uncertain	Required case data is missing, scope is ambiguous, or source interpretation is unresolved.	Route to a reviewer; do not silently exclude or assert impact.
No impact found	No configured dependency or case matched.	Report scope of analysis and data limitations; do not claim universal proof of no impact.

8. Where AI fits—and where it must not

AI can reduce the manual effort of reading long notices, but should not be the authority that changes a live compliance rule. Use AI for assistance, structured output, and triage; use validated logic and authorised review for decisions.

Task	AI role	Control
Document ingestion	OCR or parse PDF; identify headings, dates, clause references, tables.	Show extracted text and page references; allow correction.
Change suggestions	Suggest likely additions, deletions, modified clauses, and candidate rule fields.	Display old/new excerpts side by side; mark all suggestions as draft.
Dependency suggestions	Recommend potentially related forms, evidence types, and workflow steps based on registry metadata.	Require explicit confirmation; never auto-create authoritative dependencies silently.
Impact explanation	Draft a plain-language summary of why a configured dependency matched.	Ground every explanation in structured match reasons and source references.

Task	AI role	Control
Activation / legal interpretation	No autonomous authority.	Authorised human approval required; deterministic checks enforce approval gates.

Guardrails

- Never let a model silently modify approved rules, applicability conditions, deadlines, or statutory outcomes.
- Show confidence or uncertainty honestly; do not invent clause numbers, effective dates, or legal interpretations.
- Retain source snippets, page/section references, model suggestion, human edits, reviewer identity, and final approved values.
- Use role-based access, private document storage, encryption in transit/at rest, and retention policies appropriate to official records.
- If the source is unclear or conflicting, label the proposal “clarification required” and stop activation.

9. Fit with Compliance Evidence Intelligence (CEI)

RCIE becomes substantially more useful when connected to the Compliance Evidence Intelligence module. RCIE answers “what may have changed and what is affected?” CEI answers “does this application have the required evidence, and what can be verified?”

RCIE output	CEI connection
A new or revised evidence requirement	Create a proposed requirement version and map it to the evidence register after approval.
A change to accepted evidence metadata or expiry rules	Update the configured validation rule only after authorised approval; re-evaluate affected records in a controlled run.
A changed application checklist	Update the checklist configuration and preserve the version used by existing applications.
Potentially affected applications	CEI can show which cases have missing, expired, or unverified evidence under the approved new rule—without making the statutory decision.
Required follow-up	Generate targeted tasks for institutional users or reviewers, with deadlines and audit trail.

Recommended architecture

Build CEI's requirement/evidence registry and RCIE's versioned rule/dependency registry around shared stable IDs. Keep rule activation and evidence-status recalculation separate services or modules, joined by events or explicit API calls. For the MVP, a modular monolith is simpler than microservices.

10. Suggested technology stack

Layer	Recommended choice	Reason
Frontend	React + TypeScript + Vite; Tailwind CSS; a component library such as shadcn/ui	Fast to build review screens, diff views, dependency explorer, and impact dashboard.
Backend	Node.js + TypeScript + Express	Straightforward REST APIs and shared TypeScript types; suitable for a hackathon prototype.
Database	PostgreSQL / Supabase Postgres	Relational integrity, versioned records, JSON fields for scoped metadata, and audit-friendly queries.
Document storage	Private object storage with signed access	Store official source PDFs and uploaded extracts without exposing public links.
Background work	Persistent job queue or database-backed jobs	Run OCR, dependency analysis, bulk case matching, and notifications reliably.
Document extraction	PDF text extraction plus optional OCR (e.g., Tesseract or a managed service)	Support both digitally generated and scanned circulars.
Impact engine	Deterministic TypeScript service; graph traversal using relational dependency edges	Explainable, testable and sufficient for an MVP; a graph database is not required initially.
Auth & audit	Role-based access control; append-only audit events	Protect draft/approval/activation actions and provide traceability.
Optional later	Search index, graph database, event bus, advanced semantic retrieval	Add only when real scale or query complexity justifies it.

11. API sketch

These are proposed endpoint names, not an existing AICTE API specification.

```
POST /api/regulatory-sources
GET /api/regulatory-sources/:id
POST /api/change-proposals
GET /api/change-proposals/:id/diff
POST /api/change-proposals/:id/impact-runs
GET /api/impact-runs/:id/results
POST /api/change-proposals/:id/submit-review
POST /api/change-proposals/:id/decision
POST /api/change-proposals/:id/activate
POST /api/change-proposals/:id/rollback
GET /api/rules/:id/versions
GET /api/impact-runs/:id/report
GET /api/audit-events?entity_id=...
```

Example payload for a proposed change

```
{ "ruleKey": "EVIDENCE_REQUIREMENT_014", "sourceId": "SRC-2026-XX", "sourceClauseRef": "Section
X.Y (verified by reviewer)", "changeType": "modify", "oldValue": {"requiredEvidence":
["document_A"]}, "proposedValue": {"requiredEvidence": ["document_A", "declaration_B"]},
"proposedEffectiveFrom": "2026-11-01", "scope": {"applicationType": ["example_category"]},
"rationale": "Draft interpretation awaiting policy review" }
```

12. Screens and demo experience

- **Regulatory inbox:** list sources, publication/effective dates, status, and assigned owner.
- **Change comparison:** side-by-side old/new rule fields with source excerpts and reviewer annotations.
- **Dependency explorer:** rule at the centre, with linked forms, evidence requirements, validations, workflows, deadlines, and templates.
- **Impact report:** counts of direct/downstream/case-level/uncertain matches, filterable case list, and reasons for each result.

- **Approval workspace:** reviewer comments, approve/reject/request-clarification controls, and explicit effective-date confirmation.
- **Action tracker:** generated tasks, owners, deadlines, notification status, acknowledgement and completion.
- **Audit timeline:** who captured, proposed, reviewed, activated, communicated, or rolled back each change.

13. Hackathon MVP scope

Build a narrow, credible vertical slice rather than an ambitious autonomous legal AI. The judges should be able to follow one change from its official-source record through impact assessment to authorised activation.

Priority	Build item	Acceptance criterion
P0	Source register with synthetic official-style notice	Source file/URL, dates, source clause and provenance are visible.
P0	Structured rule registry with version history	Old version is preserved; draft change creates a new proposed version.
P0	Dependency mapping UI or seeded dependency data	One rule links to at least three system elements.
P0	Deterministic impact analysis	Finds seeded affected cases and explains every match.
P0	Human approval gate	Unapproved proposal cannot activate or trigger external notifications.
P0	Audit timeline	Source, proposal, analysis, decision, activation and rollback events are recorded.
P1	Task/notification simulation	Generate in-app tasks or simulated notices after approval.
P1	AI-assisted extraction	Draft clause/change suggestions with source snippets; human confirmation required.
P2	More advanced graph visualization and semantic search	Optional polish only after the end-to-end flow works.

14. Suggested 5-day build plan

Day	Focus	Deliverable
1	Schema, source register, roles and seeded example	Working source and rule-version records.
2	Dependency registry and diff interface	A proposed change with linked system dependencies.
3	Impact engine and explainable results	Repeatable analysis of synthetic applications with reasons.
4	Review/approval, activation gates, audit events	Controlled lifecycle with blocked unauthorised activation.
5	Action tracker, demo polish, test pass	End-to-end presentation, sample report, and fallback demo data.

15. Demo script (3–4 minutes)

1. Open a fictional regulatory notice in the source register. Point out its source metadata and effective date status.
2. Create a proposed change to one structured requirement and show the old/new comparison.
3. Open the dependency explorer to show linked evidence, checklist, validation and reviewer workflow components.
4. Run impact analysis. Show the candidate applications, the exact match reasons, and one uncertain case routed for human review.
5. Attempt activation as an unauthorised user and show that the system blocks it.
6. Sign in as the authorised reviewer, inspect the proposal, add a rationale and approve it.

7. Activate the new version for its approved effective window, then show generated tasks and the audit timeline.
8. Finish by explaining that the engine supports—not replaces—statutory interpretation and competent-authority decisions.

16. Test plan and evaluation metrics

Test	Expected behaviour
Rule modified	New version is created; previous approved version remains queryable.
Rule removed or superseded	Removal is represented explicitly; dependent items are surfaced for review.
Effective date is missing or ambiguous	Proposal cannot activate until an authorised user resolves the date policy.
Case lacks a scope attribute	Result is uncertain/review-needed rather than silently treated as in-scope or out-of-scope.
No dependencies configured	Engine reports “no configured dependencies found” and flags incomplete mapping risk.
Unauthorised activation attempt	Denied and recorded in the audit trail.
Notification retry	No duplicate uncontrolled notifications; delivery state and retry are recorded.
Rollback	A documented compensating version/event restores the prior intended configuration; audit history remains intact.
Conflicting source documents	System marks conflict and requires policy clarification before activation.

Useful MVP metrics

- **Impact precision:** proportion of reviewed flagged items judged truly affected.
- **Impact coverage:** proportion of known affected seeded items surfaced by the engine.
- **Explanation coverage:** percentage of results with a readable reason and full dependency path.
- **Review efficiency:** time required for a reviewer to understand and disposition a change.
- **Traceability:** percentage of active rules linked to an approved version and source reference.
- **Safety:** number of unapproved activations or unapproved external notifications—target zero.

17. Risks and mitigations

Risk	Mitigation
Incorrect AI interpretation	AI produces drafts only; require source-grounded human verification and structured rule approval.
Incomplete dependency map	Display mapping coverage; let owners confirm dependencies; treat “no impact found” cautiously.
Incorrect case applicability	Use explicit, tested scope/date rules; show matching fields and missing-data warnings.
Conflicting or superseded notices	Track source status and precedence only through authorised policy; flag unresolved conflicts.
Historical cases change unexpectedly	Version application checklists and define transition policy explicitly before re-evaluation.
Notification fatigue or misinformation	Require approval and recipient targeting; use preview, test mode, delivery tracking, and rate controls.
Security and sensitive records	Role-based permissions, least privilege, private storage, secure logs, and retention controls.
Overengineering	Use relational dependency edges first; postpone a graph database and autonomous agents until proven necessary.

18. Novelty positioning for SIH

Do not pitch the idea as merely “AI reads circulars” or “automatic regulatory notifications.” Those are narrow features. Pitch the full operational chain: source-grounded change intake, versioned rule configuration, dependency-aware impact analysis, explainable affected-case identification, human approval gates, and traceable follow-up.

What makes the proposal stronger

- **Regulation-to-workflow traceability:** each proposed operational change points back to a source clause and an approved rule version.
- **Dependency-aware analysis:** finds downstream forms, evidence checks, workflows, deadlines, and candidate applications—not just text differences.
- **Explainable impact:** each match shows why it was included and which dependency path was traversed.
- **Effective-dated governance:** distinguishes published, effective, approved, and activated states.
- **Safe change control:** preserves human authority, audit history, and controlled rollback.

Claims to avoid

- Do not claim that all legal changes can be interpreted correctly by AI.
- Do not claim complete impact coverage unless the dependency registry and case data have been validated.
- Do not claim the current AICTE platform lacks a feature without checking official documentation and the actual platform.
- Do not describe an impact flag as a legal violation, rejection recommendation, or binding decision.
- Do not use real applicant/institution data in a demo unless authorised; synthetic data is sufficient.

19. Recommended architecture and implementation decision

For SIH, build RCIE as a module in the broader ApprovalTwin platform rather than a separate product. Use React/TypeScript on the frontend, Node.js/Express on the backend, PostgreSQL for versioned rules and dependency edges, private file storage for source documents, and a deterministic impact-analysis service. Seed the demo with a small, clearly fictional rule set and synthetic applications.

Recommended MVP order

1. Rule registry + source provenance + version history.
2. Manual old/new change entry and review workflow.
3. Dependency graph stored as relational edges.
4. Deterministic impact engine with per-result explanations.
5. Approval-gated activation, audit log, and task generation.
6. Optional AI/OCR suggestions after the core workflow is reliable.

20. Final summary

RCIE makes regulatory change operationally manageable. It connects a verified source to a controlled rule version, follows configured dependencies into the approval workflow, identifies potentially affected applications with explanations, and routes actions through authorised review. The strongest hackathon demonstration is not a chatbot summarising a circular; it is a safe, auditable chain from change to impact to action.

Validation note

Before using any real AICTE rule, verify the exact text, applicability, effective date, transition arrangements, and authoritative interpretation against the current official AICTE Approval Process Handbook, circulars, amendments and relevant directions. Treat all examples in this PDF as fictional illustrations until verified.